



南開大學
Nankai University

计算机学院
数据安全课程实验报告

频率隐藏 OPE 方案实现

姓名：孔祥昊

2311439

专业：计算机科学与技术

2026 年 5 月 12 日

目录

1 实验要求	2
2 实验原理	2
2.1 核心思想：从“确定性点映射”到“随机化区间映射”	2
2.2 客户端：基于状态的随机排位	2
2.3 服务端：动态二叉平衡编码树	2
2.4 动态重编码机制	3
2.5 安全性分析：抗频率分析攻击	3
3 实验	3
3.1 复现	3
3.1.1 环境配置	3
3.1.2 代码编写及编译运行	4
3.2 插入相同数值多次的测试	5
4 Github 链接	8

1 实验要求

- 参照教材 6.3.3 FH-OPE 实现，完成频率隐藏 OPE 方案的复现，并尝试在 `client.py` 中修改，完成不断插入相同数值多次的测试，观察编码树分裂和编码更新等情况。

2 实验原理

本实验所采用的方案基于 Kerschbaum 在 CCS 2015 中提出的频率隐藏保序加密 (FH-OPE) 框架。其核心逻辑在于通过随机化技术打破明文与密文之间的一一映射关系。

2.1 核心思想：从“确定性点映射”到“随机化区间映射”

传统的 OPE 方案 (如 Boldyreva 方案) 具有确定性，即对于相同的明文 m ，始终满足 $E(k, m) = c$ 。这种特性导致密文暴露了原始数据的统计分布特征。

FH-OPE 的核心创新在于引入随机性：

- **逻辑表示**：若明文 m 在数据集中出现 n 次，方案将其视为 n 个不同的逻辑实体 $\{m_1, m_2, \dots, m_n\}$ 。
- **排序规则**：这些实体在逻辑序中保持相邻，但在物理密文编码空间上是分散且随机排列的。即便 $m_a = m_b$ ，其加密结果 c_a 与 c_b 在概率上也是不等的。

2.2 客户端：基于状态的随机排位

客户端通过维护一个本地状态表 (`local_table`) 来动态管理明文的分布信息。

在执行插入操作时，客户端通过以下逻辑计算逻辑位置 Pos ：

1. **统计计数**：检索 `local_table`，获取当前待插入明文 m 及其之前所有小于 m 的明文的总出现次数 $presum$ 。
2. **随机偏移**：设 m 当前已存在的次数为 $count$ ，则本次插入的逻辑位置 Pos 计算公式为：

$$Pos = \text{random.randint}(presum, presum + count) \quad (1)$$

该机制确保了即使是相同的值，由于随机偏移的存在，其在逻辑序列中的位置也是不确定的，从而有效地隐藏了明文的频率分布。

2.3 服务端：动态二叉平衡编码树

服务端通过用户定义函数 (UDF) 维护一棵内存状态树，用于将客户端提供的 Pos 转换为数值编码。

- **路径编码**：树的每个节点对应一个数值区间 $[L, R]$ 。根节点通常覆盖整个编码空间 (如 64 位整型范围 $[0, 2^{63} - 1]$)。
- **二分分配策略**：对于任意节点，其左子节点的区间为 $[L, \text{mid}]$ ，右子节点的区间为 $[\text{mid}, R]$ 。
- **插入操作**：服务端根据 Pos 进行树搜索，定位至目标叶子节点，并分配该区间的中间值作为物理密文。

2.4 动态重编码机制

由于编码空间是有限的，随着数据不断插入，相邻节点的编码间隙会逐渐缩小。

- **触发条件**：当某个区间 $[L, R]$ 满足 $R - L < 2$ 时，判定该空间耗尽。
- **重平衡流程**：服务端触发 Recode 流程，通过旋转树结构或重新分配子树节点的区间来扩大间隙。
- **交互同步**：服务端将更新后的编码映射表同步至数据库后端，确保索引的一致性。

2.5 安全性分析：抗频率分析攻击

FH-OPE 在安全性上实现了质的提升，主要体现在：

- **IND-OCPA 安全性**：底层采用带随机 IV 的 AES-CBC 加密，结合随机化的编码分布，使密文序列在统计上趋于均匀分布。
- **消除 1:1 映射**：通过“一对多”映射机制，攻击者无法通过统计密文频率来推断明文内容，有效防御了针对保序加密的频率分析攻击。
- **权衡与代价**：安全性的提升以存储空间（密文空间膨胀）和交互成本（动态重编码时的通信开销）为代价。

3 实验

3.1 复现

3.1.1 环境配置

1. MySQL 安装及配置

我们通过下列指令，配置 MySQL 环境。

```
1 sudo apt install mysql-server libmysqlclient-dev //安装开发环境
2 sudo mysql // 以 root 身份登录进入数据库
3 create user 'username'@'%' identified by 'password'; // 创建名为 username
   的用户，密码为 password
4 grant all on *.* to 'username'@'%'; // 给用户 username 授予全部权限
5 create database test_db; // 创建实验用数据库 test_db
```

```

kxh@ubuntu:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create user 'kxh'@'%' identified by '123456';
Query OK, 0 rows affected (0.03 sec)

mysql> grant all on *.* to 'kxh'@'%;
Query OK, 0 rows affected (0.03 sec)

mysql> create database test_db;
Query OK, 1 row affected (0.03 sec)

mysql> exit
Bye
kxh@ubuntu:~$

```

图 3.1: MySQL 环境配置

如图3.1所示, 我们成功创建了用户 kxh, 密码为 123456, 并授予其全部权限, 随后创建了 test_db 数据库。

2. python3 环境配置

我们通过下列指令配置 python3 环境。

```

1 sudo apt install python3 // Ubuntu 16.04
   以后的版本其实不需要运行这条指令, 运行了也没关系
2 sudo apt install python3-pip // 安装 pip
3 pip3 install pycryptodome pymysql // 安装 client.py 需要的包

```

至此, 我们完成了环境配置, 接下来进行代码编写及运行的部分。

3.1.2 代码编写及编译运行

创建项目文件夹 lab4-OPE, 我们将教材提供的代码复制粘贴到该文件夹下, 代码可以在最后一节访问 GitHub 链接获取, 这里不做过多展示。

我们通过以下指令编译并将文件复制到 MySQL 环境中。

```

1 g++ -shared -fPIC UDF.cpp Node.cpp -o libfhope.so //编译
2 sudo cp libfhope.so /usr/lib/mysql/plugin/ // 复制目标文件到目标目录下

```

```

kxh@ubuntu:~/lab4-OPE$ g++ -shared -fPIC UDF.cpp Node.cpp -o libfhope.so
kxh@ubuntu:~/lab4-OPE$ ls
client.py libfhope.so local.sql Node.cpp Node.h UDF.cpp
kxh@ubuntu:~/lab4-OPE$ sudo cp libfhope.so /usr/lib/mysql/plugin/

```

图 3.2: 文件示例

如图3.2所示, 在 lab4-OPE 目录下, 存在 5 个代码文件 (client.py, local.sql, Node.cpp, Node.h, UDF.cpp) 和一个编译得到的目标文件 (libfhope.so)

接下来, 我们通过以下指令操作 test_db 数据库。

```

1 mysql -ukxh -p123456 // 登录用户 kxh, 密码 123456
2 use test_db; // 切换至 test_db 数据库
3 source /home/kxh/lab4-OPE/local.sql; // 导入 local.sql

```

```
kxh@ubuntu:~/Lab4-OPE$ pwd
/home/kxh/lab4-OPE
kxh@ubuntu:~/Lab4-OPE$ mysql -ukxh -p123456
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use test_db
Database changed
mysql> source /home/kxh/lab4-OPE/local.sql
```

图 3.3: 数据库操作

最后，我们运行 client.py 得到结果如下图所示。

```
kxh@ubuntu:~/Lab4-OPE$ python3 client.py
ciphertext: KA25N8Y/HakgJ/YsdbR+0mYp2k9sSN9pm50Gi8TyhP/BcW+BN/v6FL58606075dk plaintext: banana
ciphertext: r6tpCr7g0garIcL25aaJDFvj5e6J/+mm7IeqP1Ku10c91hZP9m4uM08Uf8k8Zjf0 plaintext: orange
ciphertext: /0USzDFTeaTDAbj4P0vDHj18LQp6zdj0puZnP5m0qTe00jdyyAjTsr7M37GXxa0s plaintext: cherry
ciphertext: L7ru7EnLGFtuogj0v3owUpJ6oMo5Qz+Zj7ZMJg0e1GczEnld/P+QACrni+Z/Cxq/ plaintext: cherry
ciphertext: cmTpRxA2f7ucGRcnF1rCyJJqUo8vuAideHuXIzPEmDate/h4l+JSfWpflUv805e plaintext: orange
kxh@ubuntu:~/Lab4-OPE$
```

图 3.4: 运行结果

3.2 插入相同数值多次的测试

为完成该部分实验，我们需要重新编写 client.py 的代码，命名为 client_advanced.py

```
1 import pymysql
2 import random
3 from Crypto.Cipher import AES
4 from Crypto.Random import get_random_bytes
5 from Crypto.Util.Padding import pad, unpad
6 import base64
7
8 local_table = {}
9
10 key = get_random_bytes(16)
11 base_iv = get_random_bytes(16)
12
13 def AES_ENC(plaintext, iv):
14
15     aes = AES.new(key, AES.MODE_CBC, iv=iv)
16     padded_data = pad(plaintext, AES.block_size, style='pkcs7')
17     ciphertext = aes.encrypt(padded_data)
18     return ciphertext
19
20 def AES_DEC(ciphertext, iv):
21
22     aes = AES.new(key, AES.MODE_CBC, iv=iv)
23     padded_data = aes.decrypt(ciphertext)
24     plaintext = unpad(padded_data, AES.block_size, style='pkcs7')
25     return plaintext
```

```
26
27 def Random_Encrypt(plaintext):
28
29     iv = get_random_bytes(16)
30     ciphertext = AES_ENC(
31         iv + AES_ENC(plaintext.encode('utf-8'), iv),
32         base_iv)
33     ciphertext = base64.b64encode(ciphertext)
34     return ciphertext.decode('utf-8')
35
36 def Random_Decrypt(ciphertext):
37
38     plaintext = AES_DEC(
39         base64.b64decode(ciphertext.encode('utf-8')),
40         base_iv)
41
42     plaintext = AES_DEC(plaintext[16:], plaintext[:16])
43
44     return plaintext.decode('utf-8')
45
46 def CalPos(plaintext):
47
48     presum = sum([v for k, v in local_table.items() if k < plaintext])
49     if plaintext in local_table:
50         local_table[plaintext] += 1
51         return random.randint(presum, presum + local_table[plaintext] - 1)
52     else:
53         local_table[plaintext] = 1
54         return presum
55
56 def GetLeftPos(plaintext):
57
58     return sum([v for k, v in local_table.items() if k < plaintext])
59
60 def GetRightPos(plaintext):
61
62     return sum([v for k, v in local_table.items() if k <= plaintext])
63
64 def Insert(plaintext):
65
66     ciphertext = Random_Encrypt(plaintext)
67     conn = pymysql.connect(
68         host='localhost',
69         user='user',
70         passwd='123456',
71         database='test_db')
72
73     cur = conn.cursor()
74     cur.execute(f"call pro_insert({CalPos(plaintext)}, '{ciphertext}')
```

```
75     conn.commit()
76     conn.close()
77
78 def Search(left, right):
79
80     left_pos = GetLeftPos(left)
81     right_pos = GetRightPos(right)
82     conn = pymysql.connect(
83         host='localhost',
84         user='user',
85         passwd='123456',
86         database='test_db')
87
88     cur = conn.cursor()
89     cur.execute(
90         f"select ciphertext from example "
91         f"where encoding >= FHSearch({left_pos}) "
92         f"and encoding < FHSearch({right_pos})")
93
94     rest = cur.fetchall()
95     for x in rest:
96         print(f"ciphertext: {x[0]} plaintext: {Random_Decrypt(x[0])}")
97
98 def Repeated_Insert_Test(value, count, report_every=100):
99
100     for i in range(1, count + 1):
101         Insert(value)
102         if report_every > 0 and i % report_every == 0:
103             print(f"inserted {i} rows of '{value}'")
104             Search('a', 'b')
105
106 if __name__ == '__main__':
107     Repeated_Insert_Test('apple', 500, report_every=50)
108     Search('a', 'b')
```

结合上述代码，我们运行指令 `python3 client_advanced.py`


```

$ ./lab4-Q15 python3 client_advanced.py
Inserted 30 rows of 'apple'
cipherText: 0dadNNXpA15tN4t0nErUwy1QmKkxmg1P780Jf0dwkZ8cL0svTVmZnaXLauc+ plaintext: apple
cipherText: 0dU0l77p8CgkgepJkX0X3kx2gB084Q+VQ0TfpY0TOV/g4f915Y+R4Mq4s0Y0p plaintext: apple
cipherText: B2ecqH1F1u8v7J0Bv7ZUQ0q0vW0dR1TL6u8HfL5u8g8e8v7J0d0w4Ch3Jr/ plaintext: apple
cipherText: P1TU2y1AW08QZMT0318T0K3sVxARFE14vHzK0uXl.8BHC8hK/3t12ChtkVAr03 plaintext: apple
cipherText: n47dK53P0uz29xk8373d0PBL150Q2C5b8xrfQJ7EZ0g9vug7CMLRBP7JlD+ plaintext: apple
cipherText: 0eH1G1ZQcz2zAFNLSG8k1z0c5h+0T4u6c4G0TJ0k84rtt0qJ7Ar7JlSpL plaintext: apple
cipherText: 1+hmRDLNkX3Yg/ygPZCv0tq+g6LazK2ZK88zBFG1XhLEffrDvHkklD0xxa3m plaintext: apple
cipherText: 380xN12y0L21mb0Ad01w0m1XCDvW1W0Z050gnv150LCK5n1fYss1VnPt plaintext: apple
cipherText: 0BU1E0H1fFqH1a8F3750rW0P8rC4p11u083300Q0uLhLV0n1u0d0H1WmX155 plaintext: apple
cipherText: Vw/X3GUn7qEAbcbMwTElV0o7A2rQC0Q1N1Jpky1shZ2X1otkyrPH4GvMqncf6 plaintext: apple
cipherText: 0v4vtpP0L1Bm1fVw0uL8rny0y5hWv28kTE0300P2M113p0k0xfC+H0C plaintext: apple
cipherText: x+33J0NvHvLT4M00CLFVx0326p0t1G012PH1nZ6rUy8JwFrt5S8B0P2nK1J plaintext: apple
cipherText: 26N3DBJ259K23YBNyGASU0WUCDGF1Q5ehJeg18Ly+BuGPCU1UEn9f57H plaintext: apple
cipherText: 2Xw0Ht11g3+5V0W3772n373P0X83/JYH5+0H0FEG0L0rV4u93Y+2k4Z plaintext: apple
cipherText: Ed5mCmC/u/ZpXwS150bQk1271KTJ3XMEKsc2/+K4u48u489vVQUG4J8L plaintext: apple
cipherText: GL7TuyC8KXk1c2n1JY+H9GCUtMlg/f1/RW/kacGJ0t09+1b+TS340pX12CP9 plaintext: apple
cipherText: 0T0848gP+Kw0C1C0e/LC80733uPC0d0PntJ1kx10NkV313V0222uy plaintext: apple
cipherText: +qeEv7JgdwEPwR4KQ0u7a0V0RT58ChfSLaWd8L1AE09T5Yc50V19pN8 plaintext: apple
cipherText: MkucTcayYfacc74JTS2a05YtWCurp1RV0s0N72P0n21p0p0v083e+Q11ZuLNg plaintext: apple
cipherText: 0E2t1u08B2h03G0rT2V/Ksu42V0bby3T0w/0u0J3c1C0d1J0u0T11M4L plaintext: apple
cipherText: KE2h80P0xNUP+1tqXwGL23s25q4H41h0bKfX541YVkerf172AbhN04241 plaintext: apple
cipherText: TUXGXT21wR04Hac3MT03C0a2h0b1L0uLh3KThXERq1IawNuz0H17FM40F plaintext: apple
cipherText: x7A7Q0T73F7n5U7Z1G002B04s0d57+0U0w/Y0k4W0v11211V0v93501c1L1 plaintext: apple
cipherText: UtkVseavdyopiE4nP351B030WfFz1KV51CEMLRG+QV630a1w3G1F7ZHF1/t plaintext: apple
cipherText: 0U0959tNk38Lq4ra3QV92AdF81t1t1E7Z3CvYFFpUCh7N0uN0p2x87e/LA8L plaintext: apple
cipherText: M1510H0uP0Loa51U0N1E0dF0B103V5750K2170u0H0v0t0u1130w0dL0q1 plaintext: apple
cipherText: tVH8ZhQ4+sVhG4v53L1J774N0P0Wl.a482p0hLbZU1P3/JFl4/Qx1HmaT0 plaintext: apple
cipherText: 0D1EG0v4JhWv9p0K5Z0d62ZP4uH8E2Zp0a9H0QAL0EH4T2P9r0Z1FV5 plaintext: apple
cipherText: cto/825BhepaQ1CtQ/up02JMCVc+0sdK/euHkF0X1UuWvYB2x27h1YpXNuc plaintext: apple
cipherText: RxbZGRqJcQ1CmH1ZNYqY1uM1440Zb3J5C5YKGL16vX7QBxah1A++N50nz1VQ plaintext: apple
cipherText: 0P0K4uH8400B0m7JhC4121M0833557p15K022H4K21KZV8Fz2B7h0p5 plaintext: apple
cipherText: 04RM17W/1FKVhJTH05qutK0dNj77CFH5756L1XuhbTEhAN0dy11uVn/0w0W3X plaintext: apple
cipherText: rCXBQJEd/cAGxxL1PUPCvZ1w0ZwE00e1Pmk/08n323wgJfG1XhK55181yn plaintext: apple
cipherText: 7Bk1TnK1c4F7a7p0TbT0vK5AN0Rf0rQ0hNcL0C5F10P7K0c0S1L1V1E plaintext: apple
cipherText: 7Kqa+adtC+u0J7f1JkFh/6wJ7rNwKXrTXv0qJ1203+H0BbW1LPQMLC0CqK plaintext: apple
cipherText: 0Z1wQ8ZCQv11f8r1A1uK50P7y0kV1eH5m0N0XQ142LhC8G4tUxJ241/rC plaintext: apple
cipherText: 02010K0QhK1p11Jn00ZVx1K85e0e2J0h3Jh+0dT0e0b0uQ1w020WZ plaintext: apple
cipherText: ex813G0q70/ultZf55EQ1WwNayalQ2M2M04t19F9Ma7eTOQncr1NvEHP0p plaintext: apple
cipherText: sv0P7r0W0K08r3v0g0Mg7K7y40092C0C181P8C8Q0L0yNk7W09V1W7FZr plaintext: apple
cipherText: B1D05Vr3L0u0q0H0v003K1Q+u11C400K1151K0H30u2Mh0d0y0K0u0F0 plaintext: apple
cipherText: RFew4H0E00H7Wp5/XsuQ1uYdp161b11g/NQ77a2WY8P+Br+ss1znt2anbcg plaintext: apple
cipherText: 0Zk0h7v2M091Z655NJP1C41uH0b00700v0H5CJ132Xh0d00d1L1E100V plaintext: apple
cipherText: 3XeQLboZ1Jnp00rKf1B0K019b0323M5FC2LkTVFLP1X3q1J7Ch2W1V1K plaintext: apple
cipherText: 7YkXEBky15yBjH00F1N0G0H310d2Jf1+Pw7d76xG0L0VdLAX7M0aQ1TEC plaintext: apple
cipherText: 1t5g70t1F11F2404+K5E21X0b3FPF1VWVYK0t1Cpr4K0W3C023uE plaintext: apple
cipherText: SgV7b1B0F0P2ZcP7KCL3Jk0UJ514w4/9NCC1BhVW040bEC2KRL1M0K0D plaintext: apple
cipherText: y0ac1nK15n1J3BahUT0AYv1nT3Fa0BYR0D1W48162P7F7rJusEvAn0P+17/a plaintext: apple
cipherText: 0p10t810P0F0K0H0U1t0J3u0M1n0uR0b34L0wV7C/1AT2L5Vp5/vp/p05 plaintext: apple
cipherText: CHW0K0VH1u0qUT7R8J7t5tVW0C0B9e2K4k0Jp3W2N5deuMmWpU7L28d1 plaintext: apple
cipherText: F93W1VZV0K0qKf+nF5e0BCH71L3L0g0CL50P0C/1iatLnJyt1zn585C4K5 plaintext: apple

```

图 3.5: 前 50 次重复的结果

如图3.5所示, 对于相同的明文, 密文在重复插入多次的情况下没有出现重复的情况。

4 Github 链接

访问[Github 链接](#)获取代码。